Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs

DAY 2

Core API

CRUD, serializers, routers, viewsets, and validation

Monday, October 26, 2026

Your five-day journey

Yesterday the Task model persisted. Today it becomes an API.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD endpoints and validation

TODAY

03

Security

Tokens, ownership, isolation

04

Quality

8 tests, settings from the environment

05

Publish

Deploy, verify, README, live demo

Today's session

Learn the API pattern, practice it, then apply it to the project

Day 1 push
first 10 min

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

01 Convert Task objects to validated JSON

02 Build routed list, create, retrieve, update, partial update, and delete

03 Read the success and error status codes for each operation

04 Reject invalid input with clear field-level errors

The core API request path

Each layer has one job in a CRUD request



The router finds the action. The serializer translates and validates. The viewset coordinates the operation.

The serializer: translator and gatekeeper

days/day-02/example/serializers.py

```
class MovieSerializer(serializers.ModelSerializer):
    director_name = serializers.ReadOnlyField(source="director.name")
    added_by = serializers.ReadOnlyField(source="added_by.username")

    class Meta:
        model = Movie
        fields = ["id", "title", "genre", "release_date",
                  "director", "director_name", "added_by", "created_at"]
        read_only_fields = ["id", "created_at"]
```

The client sends director as an id. The server fills id, created_at, director_name, and added_by.

ViewSets: one class, six actions

days/day-02/example/views.py

```
class MovieViewSet(viewsets.ModelViewSet):
    queryset = Movie.objects.all()
    serializer_class = MovieSerializer

    def perform_create(self, serializer):
        serializer.save(added_by=self.request.user)
```

`queryset`
WHICH rows

`serializer_class`
HOW to convert and validate

`perform_create`
server-side values after validation

ReadOnlyModelViewSet gives only list and retrieve. POST, PUT, PATCH, and DELETE return 405.

Routers: one line, two routes

days/day-02/example/urls.py · mounted under api/ in config/urls.py

```
router = DefaultRouter()
router.register("movies", MovieViewSet, basename="movie")
urlpatterns = router.urls
```

URL	Name	Methods → actions
/api/movies/	movie-list	GET → list · POST → create
/api/movies/{pk}/	movie-detail	GET → retrieve · PUT → update · PATCH → partial_update · DELETE → destroy

Tests use these names: reverse("movie-list") and reverse("movie-detail", args=[3])

The CRUD contract

The router gives the API predictable list and detail routes

Method	URL	Action	Status
`GET`	`/api/books/`	list	200
`POST`	`/api/books/`	create	201
`GET`	`/api/books/{id}/`	retrieve	200 / 404
`PUT`	`/api/books/{id}/`	update	200
`PATCH`	`/api/books/{id}/`	partial update	200
`DELETE`	`/api/books/{id}/`	destroy	204

One POST request, step by step

A valid request creates a row. An invalid request stops before the database.

POST /api/books/ → **Router** → **BookViewSet** → **Serializer**

Valid
validation passes · perform_create saves · 201 Created + JSON

Invalid
validation fails · no row is saved · 400 + field errors

The serializer decides whether the request can reach save().

PUT and PATCH

The difference is how much of the resource the request replaces

PATCH · partial update

Send only the field that changes. Other fields stay as they are.

```
{"status": "DONE"}    → 200
```

PUT · full replacement

Send the complete object. Missing required fields return 400.

```
{"status": "DONE"}    → 400
```

A partial body is fine for PATCH. PUT needs the complete object.

Validation belongs to the serializer

Built-in rules handle common problems. Custom rules express the program's rules.

Built-in
required fields · wrong types · unique ISBN · unknown author

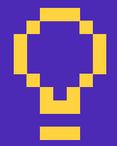
Clearer messages
extra_kwargs → "Title cannot be blank."

Custom rules
validate_<field> · self.instance tells create from update

Exercise rule
published_year cannot be later than this year

Project rule
due_date cannot be earlier than today on create

Invalid input returns 400 with field-level messages and no database change.



LIVE DEMO

The browsable API

Every endpoint, live, before you build your own

Every endpoint
list to delete

A 400
field errors

A 404
unknown id

PUT vs PATCH
same body, two results

Watch the status code and the response body every time

HOUR 2 · EXERCISE

Book CRUD API

Six operations, two validation rules, 11 passing core tests

Core	Stretch	Tests
about 50 min	if time remains	11 core to pass

Run the tests one task at a time

Exercise: what you build

Use the tests and the Movies example to guide the work

01 BookSerializer: fields, related name, read-only fields

02 BookViewSet + router: all six operations

03 Validation: clear blank-title message, year not in the future

04 Stretch: A–Z ordering with a migration, 9-request checklist

Use the hints one level at a time. The project comes before the stretch.

BREAK

Short break

After the break: apply the same pattern to the Task API

HOUR 3 · PROJECT

CRUD for the Task API

Apply the Core API pattern to your own project

Commit Day 1 → Copy checkpoint → P1–P8 → Try endpoints → Commit + push

Copy checkpoint after saving; keep Git history, .venv, and database.

Then activate .venv and run python manage.py migrate.

Day 2 project increment

Log in through the browsable API for now. Tokens arrive on Day 3.

TODO	Group	What you build
P1	Foundation	newest-first ordering; makemigrations + migrate
P2	Foundation	all 8 serializer fields, server fields read-only
P3	API surface	queryset + serializer on the viewset
P4	API surface	assign the owner on create
P5–P6	API surface	router and URLs
P7	Input rules	clear title messages
P8	Input rules	due date not in the past on create

Known gap for Day 3: every logged-in user can see all tasks today.

Day 2 exit check

Ready for Day 3 when all six pass

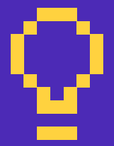
01	The router resolves list and detail routes
02	Create returns 201 and the row appears in the database
03	Retrieve returns 200; an unknown id returns 404
04	PUT and PATCH change the right fields; DELETE returns 204
05	owner is read-only and assigned by the server
06	Blank title, bad status, past due date → 400 with field errors

All ten checklist requests pass and the commit is pushed

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 2 complete

The Task API now speaks CRUD and explains bad input.

Next: token authentication, ownership, and permissions